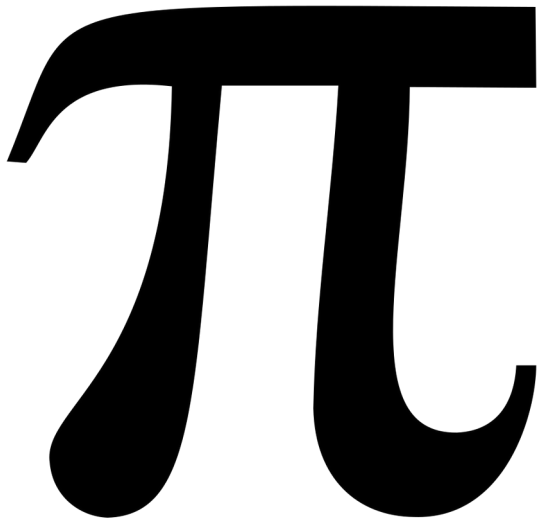
Fun Methods for Calculating



Colton Roe

Goals and Overview

- Approximate π using three methods
  - Monte Carlo
  - Buffon’s Needle
  - Collision Simulation
- Focus on the implementations rather than math
- Live Simulation / Animation for each method

Monte Carlo Sampling

- Generate a random point in a square
- Check if it's within the circle
- Repeat this for many points
- Estimate Pi using the formula:

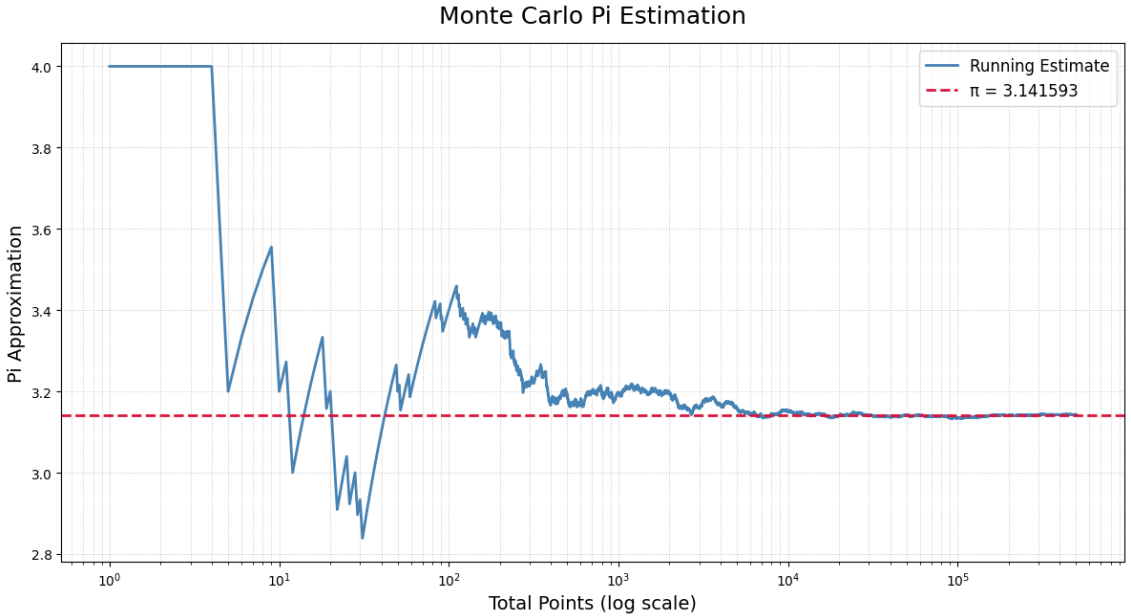
$$\pi \approx 4 \cdot \frac{\text{Number of points inside the circle}}{\text{Total points}}$$

```python
def gen_random_point():
    """Generate a random point (x, y) in the range [-1, 1]."""
    global num_points

    x = np.random.uniform(-1, 1)
    y = np.random.uniform(-1, 1)

    if is_point_in_circle(x, y):
        global points_in_circle
        points_in_circle += 1

    num_points += 1

    color = rl.BLACK if is_point_in_circle(x, y) else rl.RED
    plot_point(x, y, color)
```

```python
def estimate_pi():
    """Estimate the value of pi using the ratio of points inside the circle."""
    if num_points == 0:
        return 0
    return (points_in_circle / num_points) * 4
```

```python
def plot_point(x, y, color):
    """Map the point (x, y) to the window coordinates and draw it."""
    screen_x = int(x * (RENDER_SIZE / 2) + (RENDER_SIZE / 2))
    screen_y = int(y * (RENDER_SIZE / 2) + (RENDER_SIZE / 2))
    rl.draw_circle(screen_x, screen_y, 2, color)
```

Monte Carlo Results



- Final estimate: 3.142392 (500,000 points, 3 correct digits)
- Error: 0.0324%
- Lots of variation at low sample size
- Converges slowly with many, many points needed for more correct digits

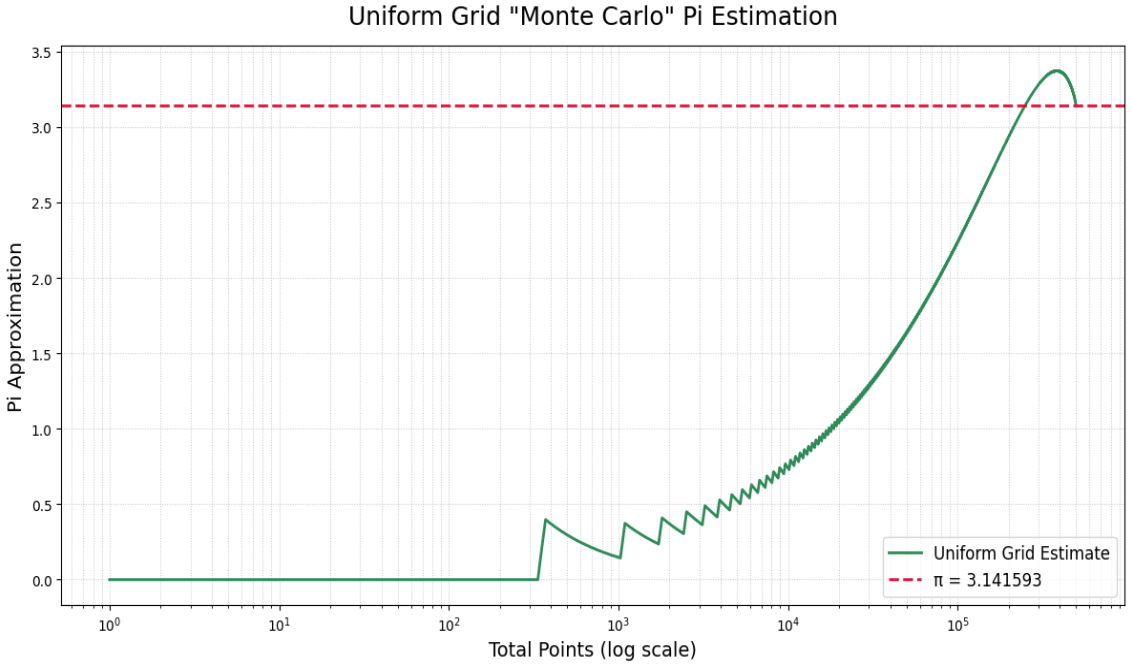
“Monte Carlo” with a Uniform Distribution

- Divide the square into evenly spaced points
- Check if each point is within the circle
- Then we use the same formula as before:

$$\pi \approx 4 \cdot \frac{\text{Number of points inside the circle}}{\text{Total points}}$$

```python
# Step size to fill a -1 to 1 square with TOTAL_POINTS points
# in a uniform grid pattern
DX = 2 / (TOTAL_POINTS ** 0.5)
DY = 2 / (TOTAL_POINTS ** 0.5)

def gen_uniform_point():
    """Generate a point (x, y) in a uniform grid pattern."""
    global num_points

    # Calculate the x and y coordinates based on the current number
    # of points drawn
    number_of_columns = int(2 / DX)
    row = num_points // number_of_columns
    col = num_points % number_of_columns

    # subtract 1 to map [0, 2] to [-1, 1]
    # then add half a step to center the point in its column/row
    x = -1 + (col * DX + DX / 2)

    # We take away from 1 instead here because in graphics libraries
    # the y-axis is flipped
    y = 1 - (row * DY + DY / 2)

    if is_point_in_circle(x, y):
        global points_in_circle
        points_in_circle += 1

    num_points += 1

    color = rl.BLACK if is_point_in_circle(x, y) else rl.RED
    plot_point(x, y, color)
```

Uniform Distribution Results



- Final estimate: 3.141504 (500,000 points, 5 correct digits)
- Error: 0.0028%
- Not accurate until all points are accounted for

Uniform Distribution Conclusion

Method	Pi Approximation	Error	Correct Digits
Monte Carlo	3.142392	0.0324%	3
Uniform Sampling	3.141504	0.0028%	5

- At 500,000 points the uniform sampling produced a more accurate estimate
- Monte Carlo has a high variance between runs due to randomness
- The uniform sampling is deterministic, it will give the same result for the same number of points

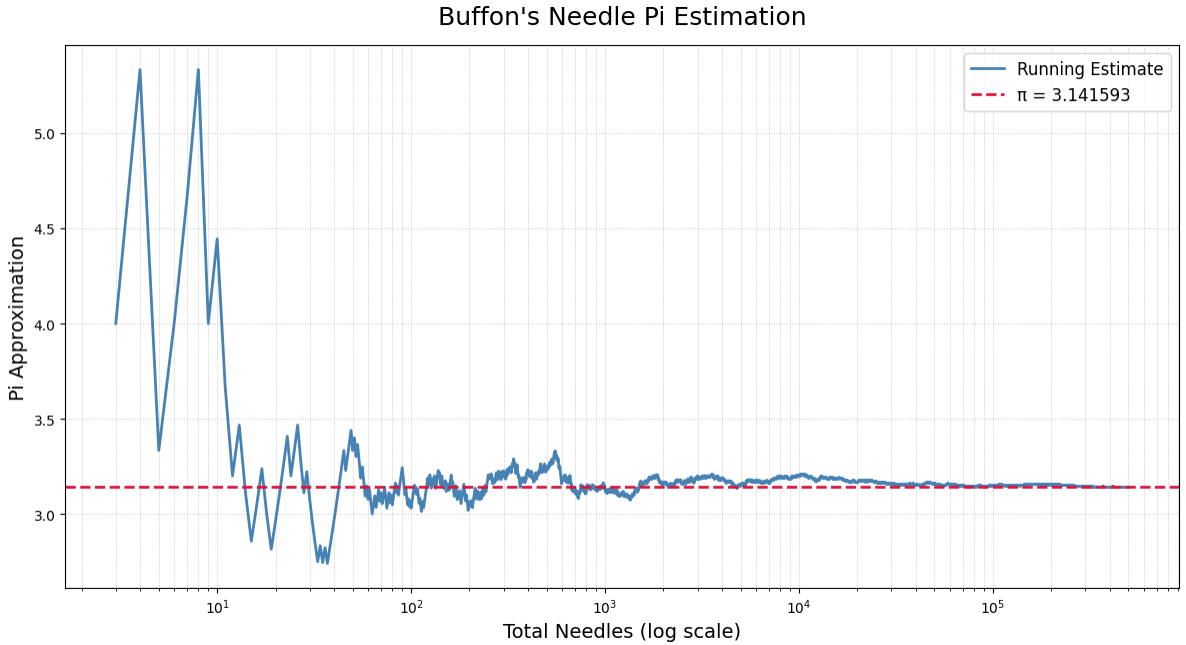
Buffon's Needle

- Drop a needle randomly onto parallel lines
- Check if the needle crosses a line
- Repeat many times
- Use crossing probability to estimate pi:

$$\pi \approx \frac{2(\text{Needle length})(\text{Total dropped})}{(\text{Distance between lines})(\text{Number of crossings})}$$

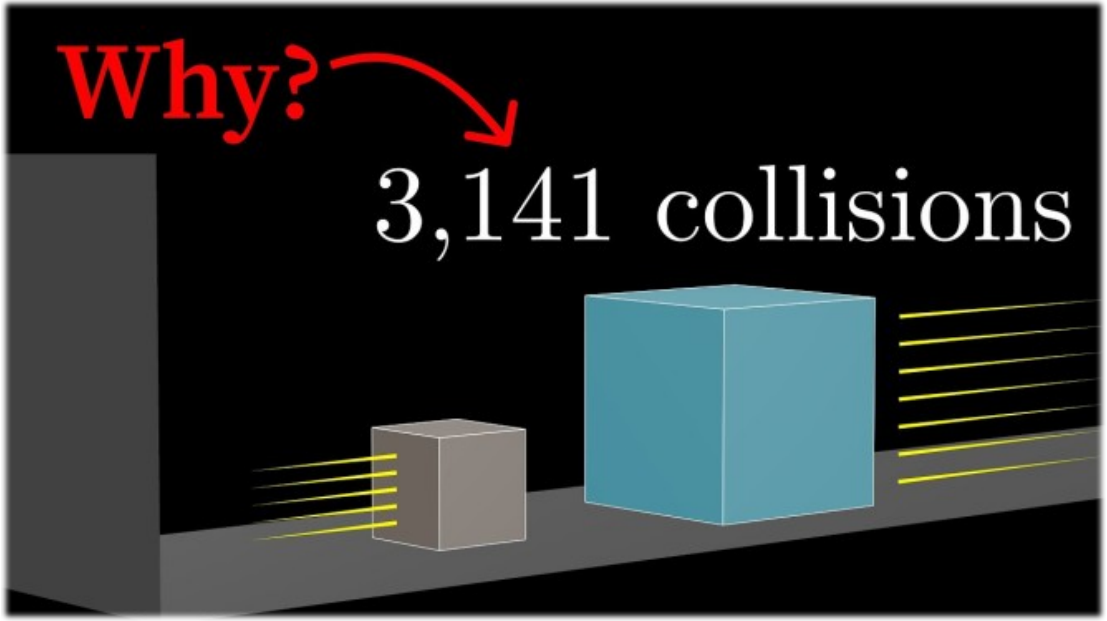
```python
def gen_random_needle():
    """Generate a random needle position and angle."""
    # The simulation area is a square
    x = np.random.uniform(0, SIMULATION_WIDTH)
    y = np.random.uniform(0, SIMULATION_WIDTH)
    angle = np.random.uniform(0, np.pi)

    global total_needles
    total_needles += 1

    if is_needle_crossing_line(x, y, angle):
        global crossing_needles
        crossing_needles += 1

    draw_needle(x, y, angle)

def estimate_pi():
    """Estimate the value of pi."""
    if crossing_needles == 0:
        return None  # Avoid division by zero

    numerator = 2 * NEEDLE_LENGTH * total_needles
    denominator = LINE_SPACING * crossing_needles
    return numerator / denominator
```

Buffon’s Needle Results



- Final estimate: 3.146851 (500,000 needles, 3 correct digits)
- Error: 0.1674%
- High variation like Monte Carlo due to random sampling

3Blue1Brown Video



- https://youtu.be/6dTyOl1fmDo?si=2Ytav6EkIzkjxDS_
- My simulation is inspired by 3Blue1Brown: “Why colliding blocks compute pi”

Collision Simulation

- Two blocks on a friction-less surface with a wall
- One block moves with constant velocity towards the other
- The blocks collide elastically until no more collisions occur
- Mass ratio controls how many digits of pi emerge

Implementation details

- Euler’s method is too inaccurate for collisions
- Use an event driven simulation instead

Implementation

- Compute the time to next collision
- If no collision: advance positions normally
- If collision occurs:
  - Jump to exact collision time
  - Apply elastic collision
- Repeat for the remaining time in the animation frame

```python
def time_to_block_collision(small, large):
    """Time until the small block's right edge meets the large block's left edge."""
    gap = large.x - (small.x + small.size)
    relative_velocity = small.dx - large.dx
    if relative_velocity <= 0:
        return float('inf')
    t = gap / relative_velocity
    return t if t > 0 else float('inf')
```

```python
def time_to_wall_collision(small):
    """Time until the small block's left edge hits the wall."""
    if small.dx >= 0:
        return float('inf')
    gap = small.x - WALL_X
    t = gap / (-small.dx)
    return t if t > 0 else float('inf')
```

```python
def advance_frame(small, large, dt):
    """Advance the simulation by dt, processing all collisions within that time.
    Returns the number of collisions that occurred."""
    collisions = 0
    remaining = dt

    while remaining > 0:
        t_blocks = time_to_block_collision(small, large)
        t_wall = time_to_wall_collision(small)
        t_next = min(t_blocks, t_wall)

        if t_next > remaining:
            # No collision before frame ends so just advance positions
            small.x += small.dx * remaining
            large.x += large.dx * remaining
            remaining = 0
        else:
            # Advance to collision, process it, continue with remaining time
            small.x += small.dx * t_next
            large.x += large.dx * t_next
            remaining -= t_next

            if t_wall < t_blocks:
                small.x = WALL_X
                small.dx = -small.dx
            else:
                elastic_collision(small, large)

            collisions += 1

    return collisions
```

Questions?